Application Overview

The technical infrustructure of FitXchange

Front-End Technologies:

- The client-facing side of FitXchange is built using HTML and Pug, JavaScript and CSS styling

Back-End Technologies:

- Node.js serves as the core back-end runtime environment.
- Docker containers are employed to encapsulate the application's environment, ensuring consistency across development.
- We have included more libraries such as express-session.

Back-End Technologies:

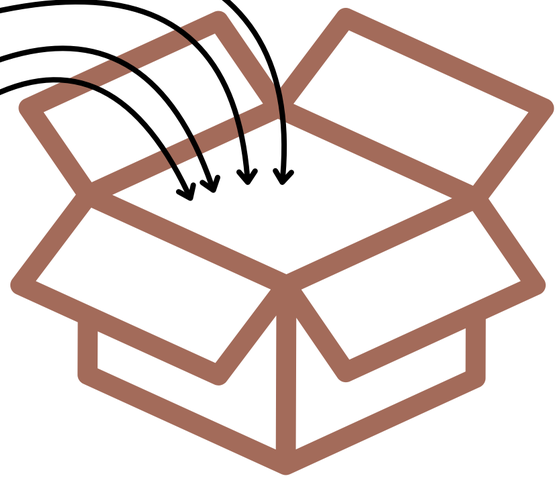
- Database Solution: MySQL, a robust relational database management system, is used to manage structured data efficiently. It supports complex queries and transactions, making it ideal for handling the intricate data relationships within FitXchange.

A User First Approach

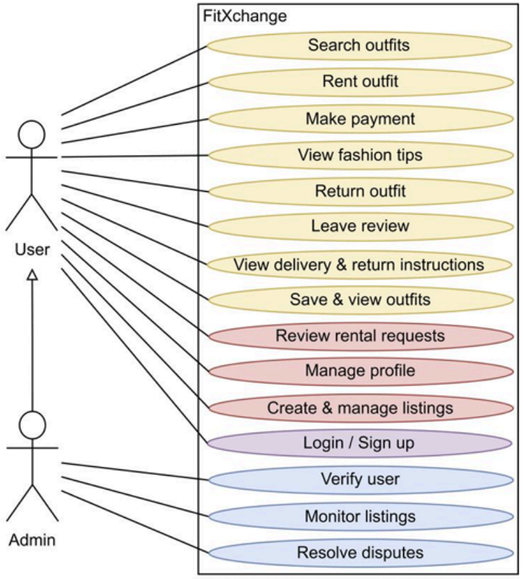
The technical infrustructure of FitXchange

Features → **User Stories** → **Use Cases**

- Borrow & Lend Outfits
- User Profiles & Customisation
- Transaction Management
- Sustainable Fashion initiative



As a user I want to...



Project Demo - A design that meets user requirements

- User Story 1
- User Story 2
- User Story 3
- User Story 4

Browsing & Filtering:
Wireframe Sections: The "Listing View" (buyer's listing view), "Fashion Advice View"

Explanation: These sections allow users to browse and filter listings, find items that match their size, and get fashion advice, which helps them identify the perfect options.

Authenticity and Quality Assurance:
Wireframe Sections: "Add Listing" (for sellers), "Profile View"

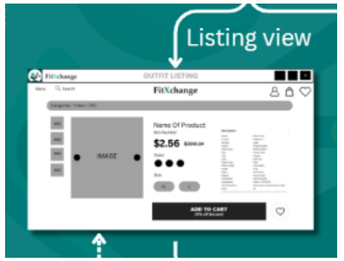
Explanation: The "Add Listing" section allows sellers to assure the authenticity and quality of their items, while the "Profile View" ensures that users can trust the information about both buyers and sellers.

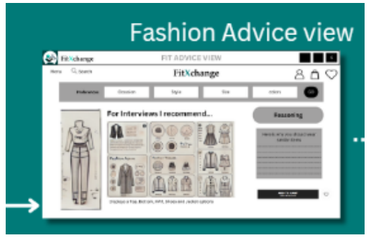
Transaction and Rental Process:
Wireframe Sections: "Checkout View", "Cart View"

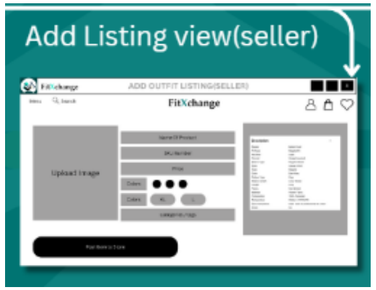
Explanation: These sections manage the transactional aspects, from adding items to the cart to completing the rental process, ensuring it's smooth and secure.

Delivery and Returning:
Wireframe Sections: "Order History"

Explanation: This section helps users track their delivery and return information, making sure they know when and how to return items.



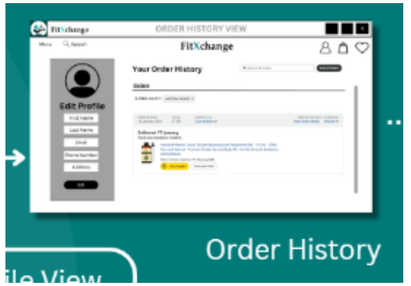












Brand Development

What technique was use to develop the look & feel of FitXchange

#083736 #6BD4D3 #9A5833 #E66EB2

#007A78 #66381F #EFBC9F #FFEC29

What Makes a successful brand?

- Provides a solution to a specific problem
- Easy to remember naming convention
- Recognizable Logos icons and elements
- Easy of use and accessibility
- Meets consumer expectations
- Consistency

What is branding & Why is it important?

- Branding is the process of creating a distinct identity for a business. It involves researching, developing, and applying features that consumers can associate with a brand's products or services.

LOGO

LOGO

Aa

Palantino Bold

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

a b c d e f g h i j k l m n o p q r s t u v w x y z

1 2 3 4 5 6 7 8 9 0

Aa

Calibri (MS)

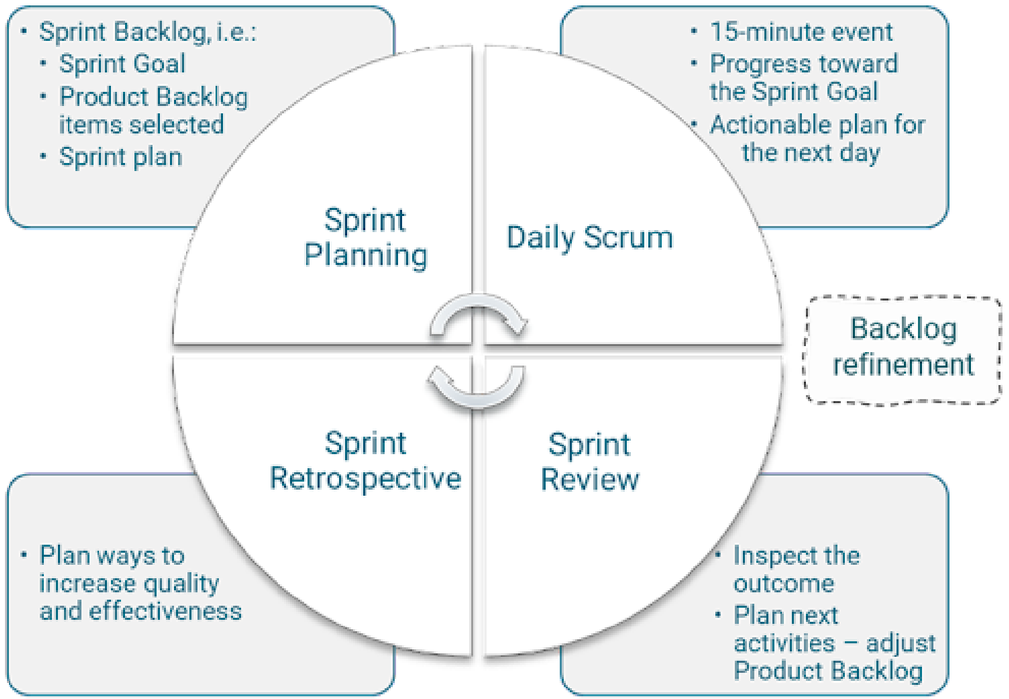
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

a b c d e f g h i j k l m n o p q r s t u v w x y z

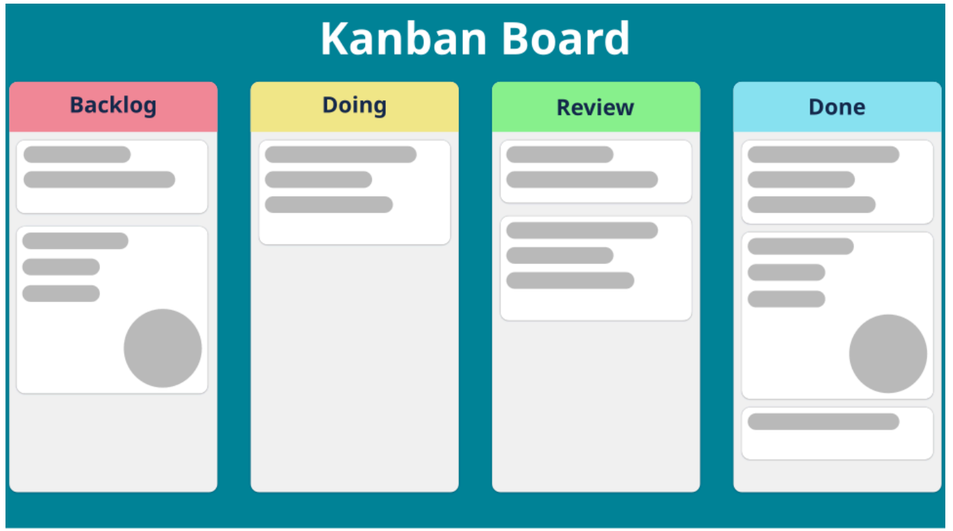
1 2 3 4 5 6 7 8 9 0

Project Management

What technique was use to manage the success of FitXchange



Agile methodology: is a project management framework that breaks projects down into several dynamic phases, commonly known as sprints.

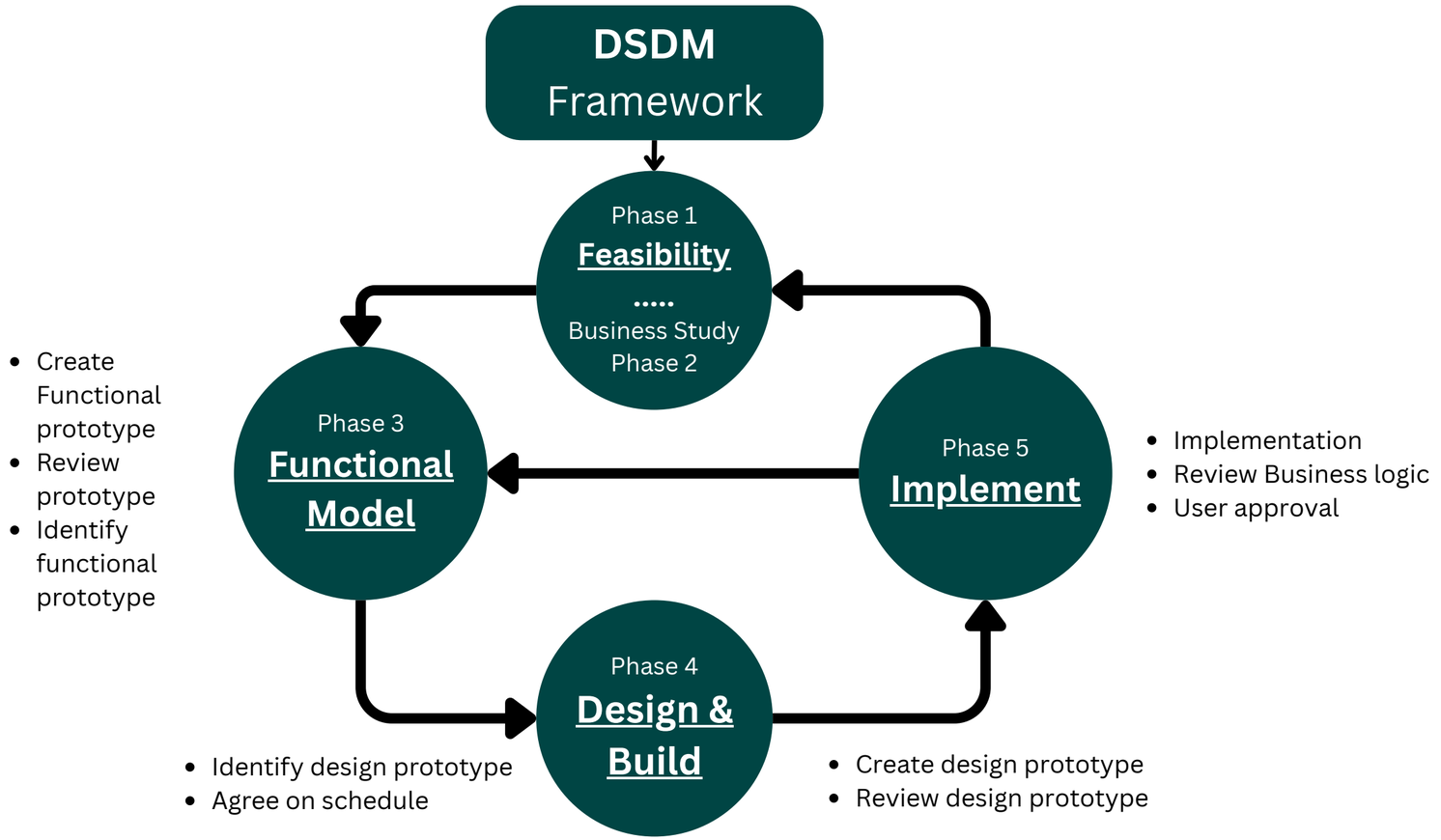


Dynamic Systems Development

Next >>

Project Management

Dynamic System Development & The FitXchange project



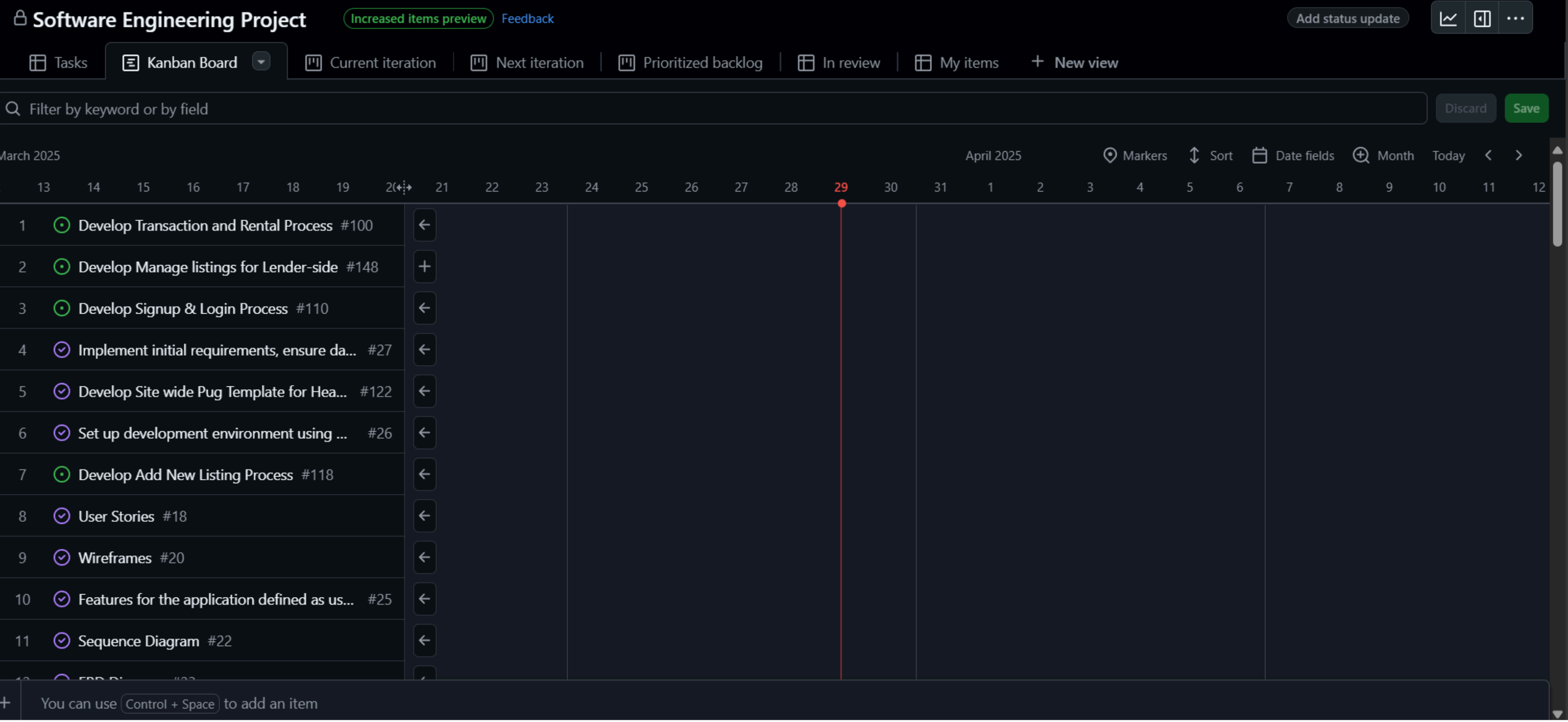
- Create Functional prototype
- Review prototype
- Identify functional prototype

- Implementation
- Review Business logic
- User approval

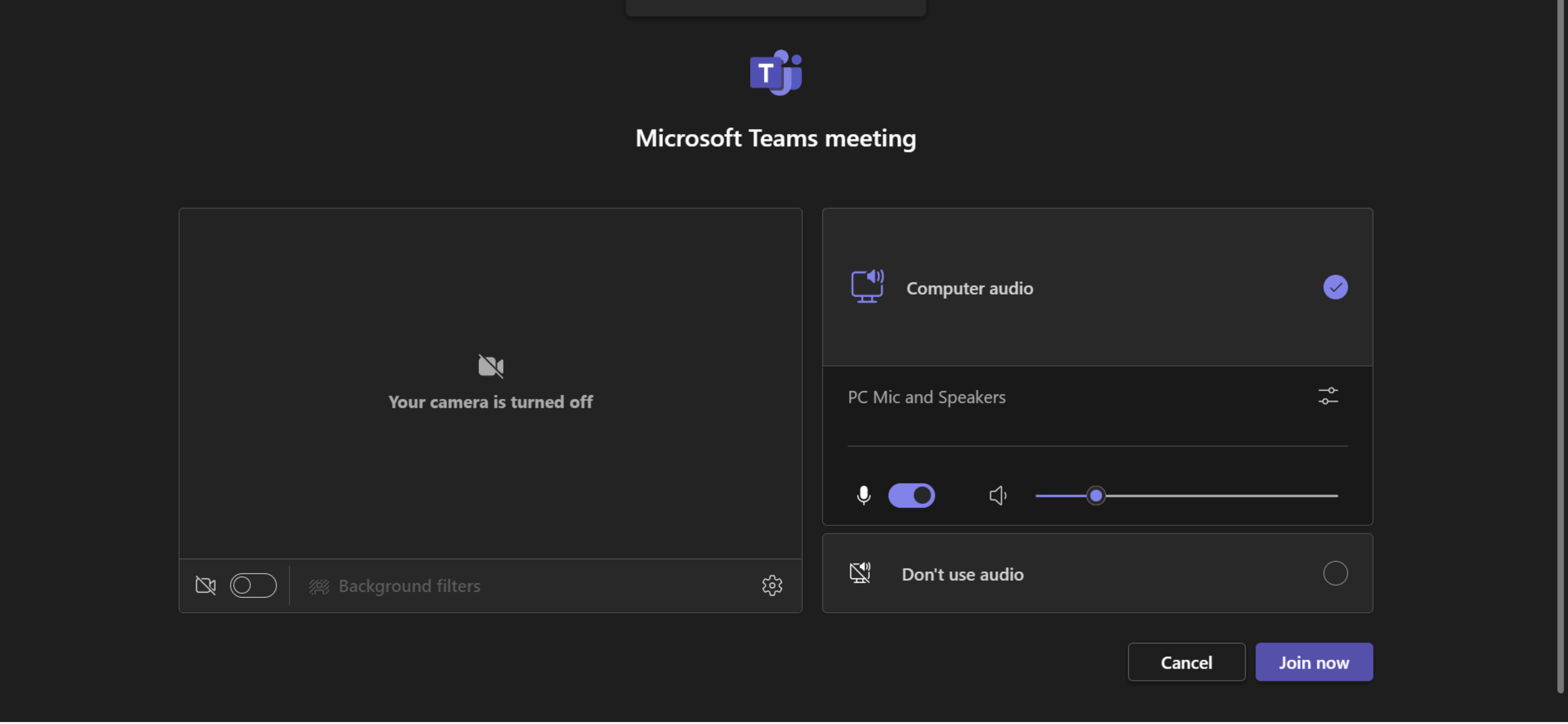
- Identify design prototype
- Agree on schedule

- Create design prototype
- Review design prototype

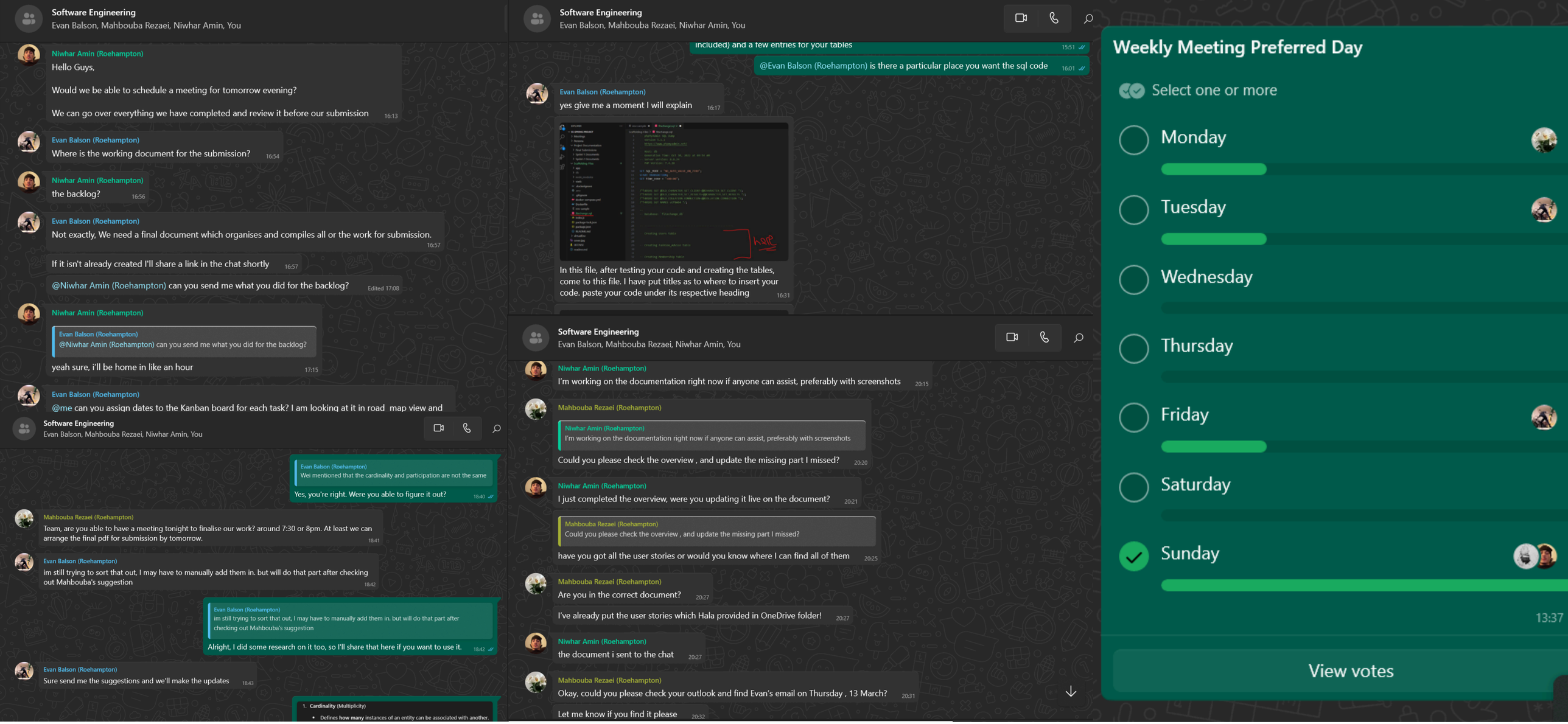
DSDM - is an Agile method that focuses on the full project lifecycle, DSDM (formally known as Dynamic System Development Method) was created in 1994, after project managers using RAD (Rapid Application Development) sought more governance and discipline to this new iterative way of working.



GitHub provides tools for managing your projects, such as issues, pull requests, and project boards. These features help you keep track of tasks, bugs, and enhancements.



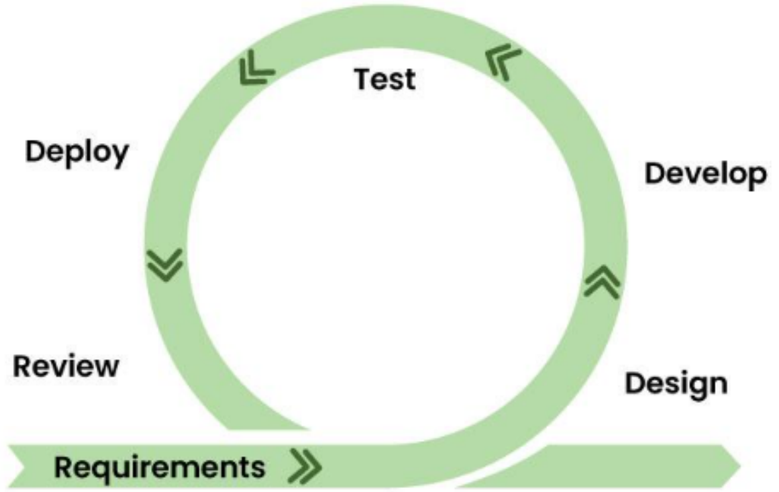
Microsoft Teams helps manage projects with channels, chats, and task integrations. It keeps teams connected, tracks progress, and streamlines communication for better collaboration.



WhatsApp makes collaboration easy with group chats, polls, file sharing, and message reactions.
These features help teams stay organized, share updates, and gather feedback quickly.

Advantages

Core advantages of our project management methodologies



Agile:

- Feature Prioritisation - Focus on the most important features first for the maximum impact
- Iterative Improvements - Continuous updates based on user feedback
- Divide & Conquer - Work is broken into small, manageable tasks for efficiency
- Team Ownership - Teams collaborate and make decisions together
- Promotes strong coordination between product owners, developers, and other stakeholders.
- Strong Coordination - Ensures seamless communication between product owners, developers and stakeholders
- Incremental Releases - Delivers small, functional updates early for quick feedback and validation
- Built-in Quality Assurance - Ensures high standards and minimises errors through development
- Risk Reduction - Continous delivery helps detect and resolve issues early

DSDM:

- Feasibility Study - Assesses business needs, constraints and viability of using DSDM
- Business Study - Defines user and data requirements, application design and maintainability needs
- Functional Model Iteration - Develops evolving prototypes based on user feedback to refine requirements
- Design and Build Iteration - Enhances prototype to ensure they deliver real business value
- Implementation - Deploys the latest working increment while allowing for further refinements as needed

The Code of Conduct

README MIT license

Code of Conduct

Team Guidelines

Respect and Inclusion

- Respect each team member's ideas and inputs.
- Foster a supportive and inclusive environment for collaboration.

Communication

- Communicate clearly and concisely.
- Keep the team updated on your task progress, blockers, or delays.

Accountability

- Provide feedback on your tasks during meetings.
- If no progress is made, state **why** and what you are doing to catch up.
- If you fall behind significantly without valid reason, it will be reported.

Meeting Etiquette

- Be punctual and prepared for meetings.
- **If you're late, treats for the team are on you!**
- **Absences:**
  - Missing **2 meetings** will result in a warning.
  - Missing **3 meetings** will be escalated and reported.

Disputes

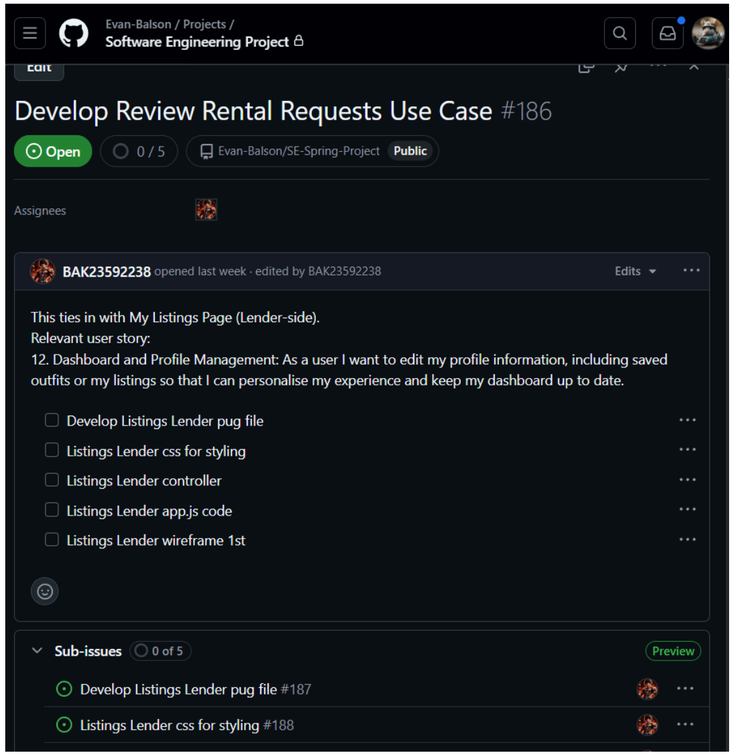
- If dissatisfied with a decision (e.g., on design features), provide proper reasoning.
- Disputes must be well-researched and presented to the team for discussion.

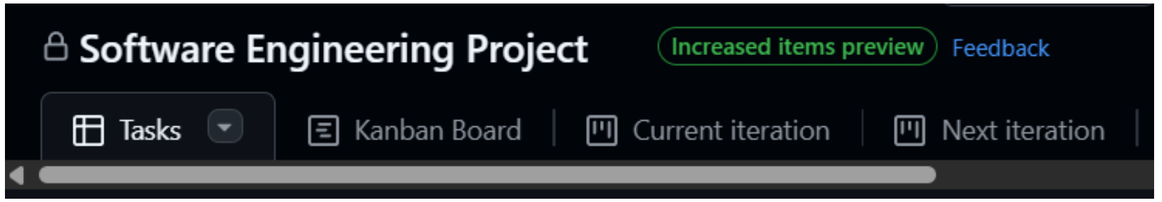
Feedback and Conflict Resolution

- Offer actionable, constructive feedback.
- Escalate unresolved conflicts to **Hala (Scrum Master)** or **Niwhar (Product Manager)**.

Github projects

- Version Control: GitHub uses Git, a distributed version control system that tracks changes to your code. Git allows multiple developers to work on a project simultaneously without overwriting each other's changes.
- Collaboration: GitHub is designed for collaborative work. It allows multiple people to contribute to a project, review code, discuss issues, and merge changes efficiently.
- Project Management: GitHub provides tools for managing your projects, such as issues, pull requests, and project boards. These features help you keep track of tasks, bugs, and enhancements



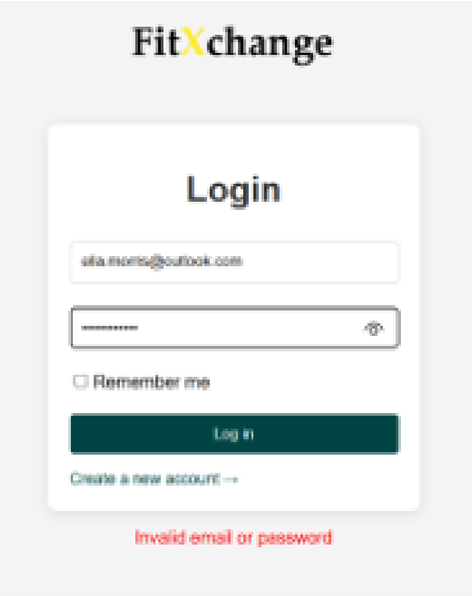


The Architecture: Model View Controller (MVC)

Core advantages of our project management methodologies

Wireframes ⟷ Server Controllers ⟷ Models

app.js | .PUG | HTML

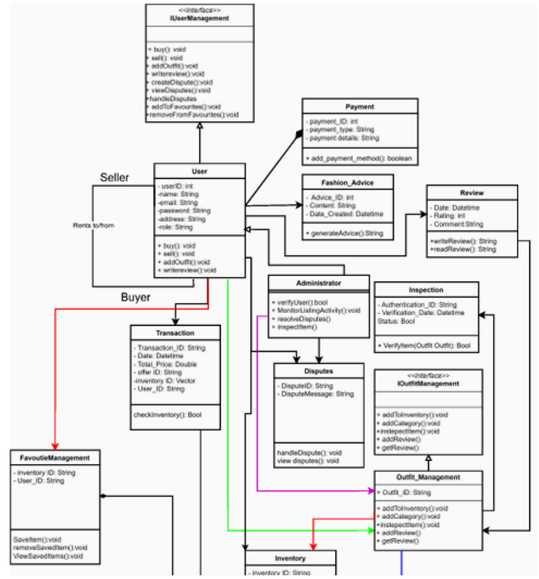



```
const { User } = require("../models/User");

exports.login = async (req, res) => {
    try {
        const { email, password } = req.body;

        // Authenticate user with the provided e
        const user = await User.authenticate(ema

        if (user) {…
        } else {
            // Invalid login
            res.render("login", { title: 'Login'
        }
    } catch (error) {
        // Log and send error message
        console.error('Error during login:', err
        res.status(500).send('Internal Server Er
    }
};
```



The Architecture: Model View Controller (MVC)

Core advantages of our project management methodologies

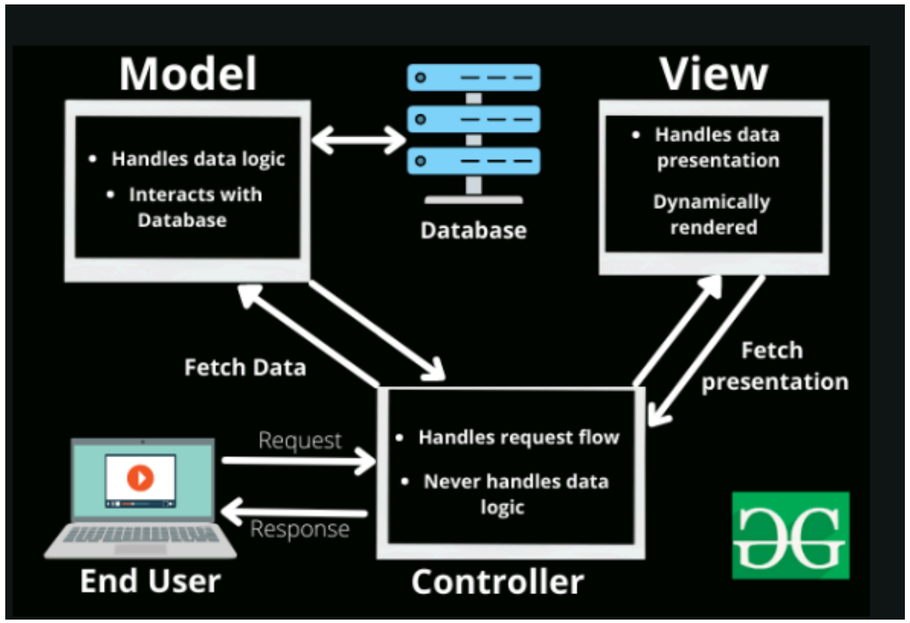


Image from Geeksforgeeks.com

Advantages

- Easy Maintenance & Extensibility - Add new features like advanced search or payment integration without disrupting the platform
- Independent Testing - Test features (e.g., profiles, payments) separately for better reliability
- Faster Development - Backend, UI, and logic can be developed simultaneously
- Reduced Complexity - MVC's separation of concerns keeps the codebase organized
- Supports TDD - Ensures high-quality, secure features through automated testing
- Optimised for Web Apps - Facilitates collaboration across teams of designers and developers
- SEO-Friendly - MVC structure improves search visibility, driving traffic to FitXchange

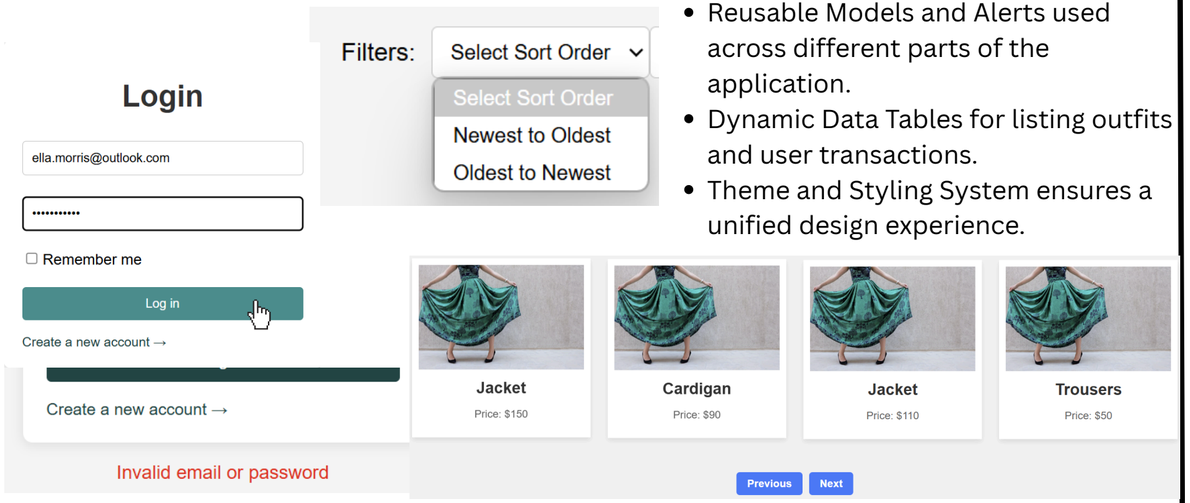
The Architecture: Reusable Components

Core advantages of our project management methodologies

FitXchange uses reusable UI and backend components to ensure consistency, reduce redundancy, and simplify maintenance. This allows for faster development, easier updates, and a scalable system.

UI Components

- Button, Form, and Input Components
- Reusable Models and Alerts used across different parts of the application.
- Dynamic Data Tables for listing outfits and user transactions.
- Theme and Styling System ensures a unified design experience.



Backend Components

- Authentication Module handles user login, registration, and authorization.
- Database Models, defines reusable data structures for users, outfits, and rentals.
- API Endpoints, modular API routes for easy expansion.
- Payment and Notification Services, separated modules for handling transactions and alerts.

UserLoginController.js

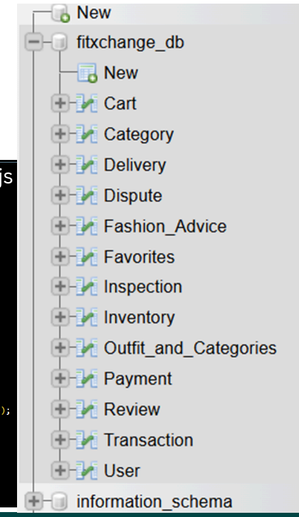
```
const { User } = require("../models/User");

exports.login = async (req, res) => {
    try {
        const { email, password } = req.body;

        // Authenticate user with the provided email and password
        const user = await User.authenticate(email, password);

        if (user) {
            // Store user info in session
            user.setLoginStatus(true);
            req.session.activeUser = user;

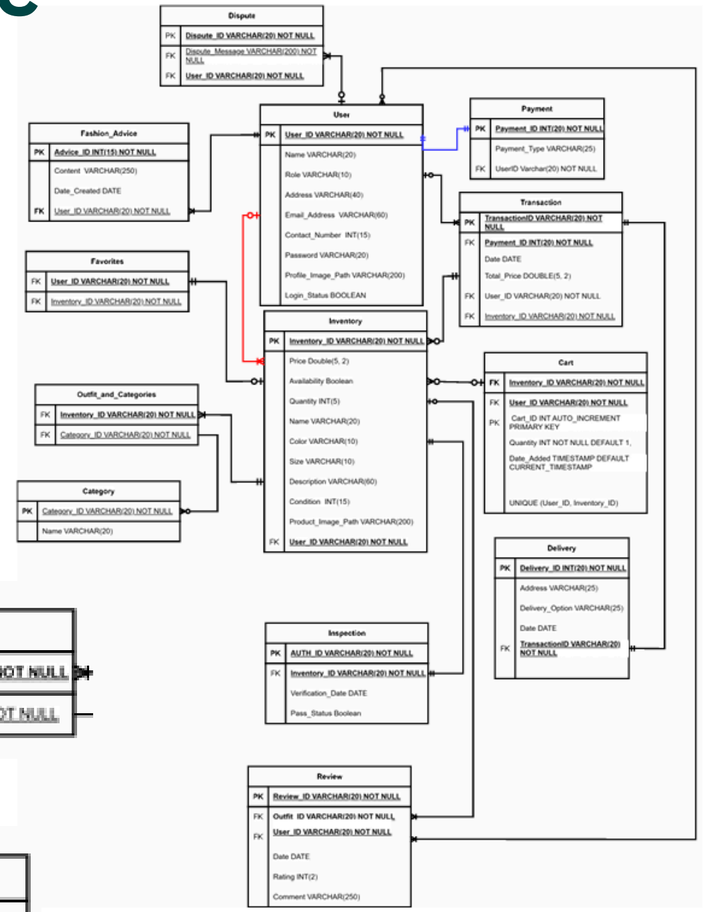

            // Redirect to home page (root)
            return res.redirect("/");
        } else {
            // Invalid login
            res.render("login", { title: 'Login', error: 'Invalid email or password' });
        }
    } catch (error) {
        // Log and send error message
        console.error('Error during login:', error);
        res.status(500).send('Internal Server Error');
    }
};
```

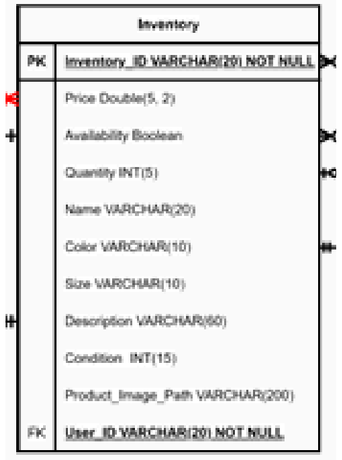


The Architecture: The Database

Core advantages of our project management methodologies

- MySQL, a robust relational database management system, is used to manage structured data efficiently. It supports complex queries and transactions, making it ideal for handling the intricate data relationships within FitXchange. We have made significant updates to the initial design to allow for more streamlined relational database.
- We chose MySQL for its reliability and scalability, supporting complex relationships while ensuring efficient performance as FitXchange grows.
- The diagram represents key tables and their relationships, such as how Outfits connect to Inventory for stock management, and Users are linked to Orders for transaction tracking.









Implemented security Measures

Core advantages of our project management methodologies

- **Error handling with Try & Catch**, prevent application crashes to improve user experience.
- **Secure password storage with Hashing** to protect user credentials from breaches.
- **Single Responsibility Principle (SRP)** to ensure each class has one purpose, to make code mere maintainable and scalable.
- **SQL Parameterization for Security**, use prepared statements to prevent malicious attacks to ensures that user input is treated as data, not executable SQL code.



Data Privacy and Security

Issue: Personal data, including contact and payment details, may be compromised if not properly secured.
Solution: Implement strong data protection measures to prevent unauthorized access and misuse of personal information.

Hash Function #SHA-2

Enhanced Security: By hashing user data with SHA-2, we ensure that even if our database is compromised, sensitive information cannot be accessed or decoded without the corresponding decryption key.

By implementing these advanced security measures, we prioritize both the privacy and safety of our users, ensuring a more secure and trustworthy environment for all.



CI workflows & GitHub Actions

Continuous integration and it’s justifications

Continuous Integration (CI) is a DevOps practice where developers regularly merge code changes into a central repository. Automated builds and tests are then triggered to ensure the software is always in a deployable state.

Why CI is Necessary:

- Stable Code - Frequent integration helps detect issues early and keeps the software functional
- Quick Feedback - CI provides immediate feedback, making it easier to fix issues fast
- Supports Agile - Enables frequent, smaller releases, which are key for agile workflows
- Improves Collaboration - Automates testing and builds, keeping the team aligned

Justification:

- Automation - Speeds up delivery by automating builds, tests, and reporting
- Quick Issue Detection - Immediate feedback helps catch issues early
- Continuous Improvement - Streamlines testing and integration, reducing waste
- Frequent Releases - Supports Continuous Delivery for regular, reliable updates

CI workflows & GitHub Actions

Continuous integration and it’s justifications

